

About the Course

Overview

The primary goal of this course is to give you an understanding of how programming languages work and provide you with the skills to assess languages based on their choices of implementation details.

The Plan

- Context/Foundations
- Syntax/Semantics
- Programming Language Features
- Programming Language Paradigms
 - Object Oriented
 - Functional
 - Logic

Grading

Component	Weight
Assignments	68%
Exams	28%
Misc	4%

Resources

Required textbook: *Concepts of Programming Languages* 12/E (Sebesta)

The textbook will be provided to you via the bookstore’s “inclusive access” program which *automatically bills you*, but at a much cheaper price than you will find elsewhere.

Details are on Canvas.

Resources II

You are expected to read **everything** that is assigned (and required). If you don’t:

- Do plan on failing.
- Do not plan on sympathy.

Help

Ask questions in class

I am available during my office hours and online throughout the day via email and the #csc415 slack channel.

Additional office hours can be planned in advance, so contact me!

Ask questions in class

Help II

Ask questions in class

Help III

Ask questions in class

Important Class Policies

- Late submission penalty: **50%**
- Plagiarism will be punished as severely as possible

Staying Up To Date

Canvas will be used to handle the general organization of the course and all **critical** announcements.

Other reminders, notes, etc. may be distributed via twitter @msupwright4 and Slack.

Final Note

My goal is to push you hard. Easy classes are not worth the money you are paying and a degree with no actual skills is worthless.

Be responsible. Ask questions. Do your work.

Overview

Programming Languages

When we are discussing programming languages, we can consider a few core, high level aspects:

- Syntax
- Semantics
- Implementation

Syntax

The first thing you will likely notice about a language is its **syntax**. The syntax of a language is, basically, *how programs in that language are written*.

In other words, what does the code *look like*?

Syntax II

While many people focus on the syntax when learning a language, its actually a very superficial aspect of the language, because understanding the **syntax** doesn't tell you anything about what the program *does*.

Semantics

If we want to know what a program actually does, we have to discuss its **semantics**.

In other words, how does the **syntax** map to the actual behavior of the programming language.

Semantics II

Unfortunately, it is very difficult to express the semantics of a programming language. Language specifications very often contain ambiguities with respect to the semantics, so its often the case that the **semantics** are only fully described by the **implementation** of a language.

Implementation

The **implementation** of a language describes how we actually “run” the code, or, more concretely, how you write the programs that “run” the programs.

Implementation II

The focus of this course will be on the *syntax and semantics* of various programming languages, though we will be discussing some implementation details along the way.

Foundations

In order to make sure that we have the proper context for the discussions in this course, we must first cover some foundational topics about how computers work in general and, at a high level, how programming languages “run.”

Von Neumann

Most computers are based on the **von Neumann Architecture**. Such computers have a central **processor** which follows the following simple algorithm:

- Fetch instruction from memory
- Optionally load data from memory
- Execute the instruction
- Determine the location of the next instruction

Von Neumann II

It’s important to note that both **instructions** and **data** are stored in the same memory, and that both consist of nothing other than bit patterns (colloquially: “zeroes and ones”).

Von Neumann III

In other words, the instructions and data, *in memory*, have no special meaning. They are simply numbers stored in memory.

The meaning of the instructions and data only exists in the sense that the instructions have been written by us to act in a certain way.

Von Neumann IV

For example, we might have 24 bytes in memory that, in isolation have no real meaning, but if we set the **program counter** to point to the first 8 bytes, the CPU might **decode** those bytes as an **integer addition** instruction.

Von Neumann V

The instruction may also reference the next 8 bytes as one operand and the last 8 bytes as another. Only at this point is it clear what these 24 bytes represent.

Data Types

However, the processor only uses a very limited number of **data types**. For instance, it may only understand integer and floating point values.

It may only provide operations for these data types and operations for loading and storing data in memory.

Programming Languages II

At this point, we reach what we might describe as the ultimate purpose of a programming language (and implementation):

Provide humans with a high level way to model problems, and the ability to translate those models to the simple set of instructions implemented by a computer.

Assembly Language

If we want to work at a higher level of abstraction than bit patterns in memory, we might use an **assembly language**.

An assembly language can be thought of as an *almost* one-to-one mapping from easy to remember instruction names to their corresponding bit patterns.

Assembly Language II

For instance, we could write something (sort of) like:

```
SUBI R1, SP, 32
LOAD R2, R1
SUBI R1, SP, 0
```

```
LOAD R3, R1
ADDI R1, R2, R3
SUBI R2, SP, 64
STOR R2, R1
```

Which would then be *assembled* to the corresponding bit patterns which the processor can understand.

High Level Languages

While this is definitely easier than writing the bit patterns directly, we are still forced to work at the low level of the processor with all of its limitations.

High Level Languages II

The content of this course is largely related to how we can instead write something like:

```
int add(int x, int y){
    return x + y;
}
```

And translate it to (more or less) the same code.

Translation

We have now mentioned translation a few times, so let's talk about what that entails.

At a high level, it simply describes the process of taking a program written in one language and turning it into an *equivalent* program in some other representation.

Compilation

In many cases, we will be **compiling** our code in order to affect a translation from the original language to some other representation *before program execution*.

Compilation II

For instance, we might create a c++ program in a file called *hello.cpp*, then execute:

```
g++ hello.cpp -o hello
```

which will take our code and translate it to bit patterns which can be directly executed by the processor.

Interpretation

Another approach which is commonly taken is **interpretation**.

In this case, we write an **interpreter** which reads our code **during execution** and tells the computer how it should behave.

Interpretation II

How does it tell the computer how to behave, though? In reality, it does not. Instead, when the **interpreter** itself is **compiled**, the bit patterns for every operation the interpreter can execute are generated and stored as part of the interpreter code.

Interpretation III

When some operator gets *interpreted*, the interpreter simply sets the *program counter* to the first instruction in the group of instructions necessary to compute the result.

JIT Compilation

An increasingly common approach is the combination of the above approaches which is often referred to as **Just in Time Compilation**.

JIT Compilation II

In this case, the interpreter used has the ability to identify code that is being repeatedly interpreted and **compiles** the code, *during execution*, to the native bit patterns needed by the processor.

JIT Compilation III

So, instead of interpreting operation after operation, identifying and calling the pre-compiled native code, the interpreter will compile, perhaps, an entire function into a new location in memory.

JIT Compilation IV

Every time the function is called in the future, the interpreter will set the *program counter* to point to those instructions instead of interpreting the function step by step.

Even More Layers

If you are using Java on a modern processor, the situation may be even more interesting. The path your code takes on the way to its actual execution is as follows:

Even More Layers II

- Java is *compiled* to Java byte code
- The byte code is *interpreted* by the Java virtual machine
- The JVM performs JIT compilation on much of your code

Even More Layers III

The precompiled code in the interpreter and the JITed code are bit patterns for the processors **instruction set architecture**, but the core of the processor may implement its own set of “micro codes”! Which means that we have one last step:

- The processor interprets the “native code” and executes it as another set of micro code instructions.

Even More Layers IV

If you happen to be executing within a virtual machine using Virtual Box or something similar, then there may be even more layers of interpretation/compilation that occur!

Zooming Back Out

Now that we have taken a quick dive into the above implementation details, we are now in a good position to better understand how we get from a high level language to the “bare metal” and how the decisions made by language designers and implementers can play a critical role in this process.

Describing Syntax

Describing Syntax

When writing a program in a given language, it is necessary that our program is valid according to the *syntax* of the language.

Describing Syntax II

For this reason, it is very important that we be able to describe what the valid syntax for the language is.

Describing Syntax III

A succinct, precise definition of the valid syntax can then be used by both *humans* writing programs and *programs* processing those programs.

Describing Syntax IV

To create such a description for a programming language we will use *context free grammars*.

Review

First, though, we must review some theoretical material which you have hopefully seen in the past.

(though it's ok if you haven't!)

Sets

A set is an **unordered** collection of elements.

We can notate a set like $\{a, b, c\}$.

We indicate that an element a is a **member** of the set A by writing $a \in A$, otherwise we write $a \notin A$.

Strings

A **string** is, essentially, a sequence of elements from a given **alphabet**.

An **alphabet** is just a **set** of symbols.

The symbols do not have to be characters, though they typically will be in this context.

Languages

A **language** is a **set of strings** over a given alphabet.

For example, given the alphabet $A = \{a, b, c\}$, we can define the language of **all strings** over A as:

$\{\Lambda, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, \dots\}$

Where Λ is the symbol for an *empty string*.

What We Want To Do

As mentioned previously, there are typically two problems we are concerned with considering the syntax of a programming language:

- Describing the proper syntax to programmers
- Recognizing programs which use proper syntax

What We Want To Do II

In the context of the definitions above, we can redefine these two syntax problems as:

- Describing how to generate the strings in a language
- Recognizing strings that belong to a given language

What We Want To Do III

These two problems are, more or less, the inverses of each other, and we will see that **grammars** are at the core of both.

Language Processors

For language processors, we are mainly concerned with the second problem: recognizing strings belonging to a given language.

Java

We could, for example, define the language *JAVA* as the set of every valid Java program.

How could we naively recognize valid strings in this language?

Java II

Most likely, you have learned to **generate** valid programs by looking at a lot of examples and having the compiler yell at you over and over until you learned some ad hoc rules:

- Methods must be contained inside of a class
- Methods can not be nested
- Types come before identifiers
- ...

Java III

When trying to recognize valid programs, you have probably relied on those same rules (and the compiler's complaints).

Using **grammars**, though, We can actually formalize most of those ad hoc rules and then they can be used by programmers to understand how to write valid programs and language processors to check the validity of programs.

Grammars

A **grammar** G for a language L over the alphabet A consists of the following:

- a set of **terminal symbols** taken from A
- a set of **non-terminal symbols** disjoint from the alphabet A
- a start symbol in the set of *non-terminal* symbols
- a set of **productions** which describe how a **nonterminal** symbol can be transformed into other symbols.

Terminal Symbols

The *terminal* symbols are the valid symbols which we expect to find in an actual, valid string in the language L .

Non-Terminal Symbols

The *non-terminal* symbols are other symbols which are solely used in the description of the language.

These symbols will never appear in a valid string in the language L .

Start Symbol

The *start symbol* is the non-terminal symbol that we begin with when we generate a string in the language L .

Productions

The *productions* of the grammar describe how *non-terminal* symbols can be turned into other symbols (*non-terminal* and *terminal*).

Productions II

To get a valid string from the language L , we will take the start symbol, apply a production symbol, and derive a new string of symbols.

Productions III

We then repeat the process of picking a non-terminal, applying a production, and replacing it with a new string of symbols.

We continue with the process until we have no more non-terminals in our derived string.

Productions IV

For example, in a grammar describing arithmetic expressions, we might have the following production:

$$Expression \rightarrow Expression + Term$$

We can infer that an **expression** consists of an **expression** followed by the operator ‘+’ followed by another **term**.

Productions V

We can also have more than one production for a given nonterminal symbol:

$$\begin{aligned}Expression &\rightarrow Expression + Term \\Expression &\rightarrow Expression - Term \\Expression &\rightarrow Term\end{aligned}$$

Productions VI

We can use the following construct to shorten such cases:

$$\begin{aligned}Expression &\rightarrow \\Expression + Term &| Expression - Term | Term\end{aligned}$$

Productions VII

To complete this grammar, we would also have the following:

$$\begin{aligned}Term &\rightarrow Term * Factor | Term / Factor | Factor \\Factor &\rightarrow (Expression) | Integer\end{aligned}$$

Productions VIII

We could further add:

$$IntLit \rightarrow Digit | Digit IntLit$$

Though we will see shortly that we will usually define the grammar for a language without that level of detail.

Extended Backus-Naur Form

Backus-Naur Form

In the context of programming languages, you will often see context free grammars written in *Backus-Naur Form* (BNF).

Non-Terminals

In BNF, non-terminals are denoted using angle brackets. For example, when defining the grammar for arithmetic expressions, we would write

$$\langle expression \rangle$$

to identify the non-terminal for expressions.

Terminals

Terminal symbols in BNF are denoted as literals with no special markup. For instance, we might have the following production

$$\langle expression \rangle \rightarrow \langle expression \rangle + \langle term \rangle$$

which includes the terminal $+$ operator.

Arithmetic Expressions

Accordingly, the BNF representation of the grammar for arithmetic expressions would be:

$$\begin{aligned} \langle expression \rangle \rightarrow & \langle expression \rangle + \langle term \rangle \\ & | \langle expression \rangle - \langle term \rangle \\ & | \langle term \rangle \end{aligned}$$

Arithmetic Expressions II

$$\begin{aligned} \langle term \rangle \rightarrow & \langle term \rangle * \langle factor \rangle \\ & | \langle term \rangle / \langle factor \rangle \\ & | \langle factor \rangle \\ \langle factor \rangle \rightarrow & (\langle expression \rangle) \\ & | \langle integer \rangle \end{aligned}$$

Extended BNF

Some extensions have been added to BNF to make defining grammars more succinct. Note, though, that this does not make the grammars more *powerful*.

Repetition

Previously, we looked at a production similar to:

$$\langle expression \rangle \rightarrow \langle expression \rangle + \langle term \rangle \\ | \langle term \rangle$$

This production uses *recursion* to express, basically, that we can have multiple operators.

Repetition II

Recursion, though, is often difficult to understand and complicates working with such productions.

In EBNF, we can replace it with

$$\langle expression \rangle \rightarrow \langle term \rangle \{ + \langle term \rangle \}$$

where the braces indicate that the enclosed part can appear *zero or more times*.

Optional Parts

If we, hypothetically, only wanted to allow binary expressions, we could write

$$\langle expression \rangle \rightarrow \langle term \rangle [+ \langle term \rangle]$$

where the square brackets indicate the enclosed part can appear *zero or one time*.

Alternatives

We only handled addition in the above example, but expressions with subtraction are *very* similar.

We can extend the earlier production to accomodate both operations with

$$\langle expression \rangle \rightarrow \langle term \rangle \{ (+ | -) \langle term \rangle \}$$

where the parentheses indicate alternatives for that part of the grammar.

Arithmetic Expressions III

Using these extensions, we can reduce the grammar to

$$\begin{aligned}\langle expression \rangle &\rightarrow \langle term \rangle \{ (+|-) \langle term \rangle \} \\ \langle term \rangle &\rightarrow \langle factor \rangle \{ (\times|/) \langle factor \rangle \} \\ \langle factor \rangle &\rightarrow (\langle expression \rangle) | Integer\end{aligned}$$

Limits

Another extension that you might see is the use of additional superscripts to indicate limits on repeated elements.

For instance, $\{+\langle term \rangle\}^5$ would indicate that you can repeat the part five times.

Also common is $\{+\langle term \rangle\}^+$ indicates that the part *must occur at least once*.

Standard

Note that there is an official standard for EBNF, but in practice people tend to make their own little tweaks to EBNF. This means that, in practice, you may see grammars that don't match what we have covered here. They *should be similar enough to understand*, though.

Attribute Grammars

Attribute Grammars

The main purpose of a grammar is to define the *syntax* of a programming language. However, we have already seen that some *semantics* can be encoded (like operator precedence) by cleverly designing the grammar.

Attribute Grammars II

With a small extension to a basic context free grammar, we can encode *even more semantics*. These extended grammars are called *attribute grammars*.

Attribute Grammars III

The basic idea is to extend the grammar by adding **attributes** to the nodes of the parse tree and information about how you could compute those attributes.

Attribute Grammars II

There are two types of attributes which we can use:

- Synthesized Attributes
- Inherited Attributes

Synthesized Attributes

Synthesized attributes are attributes which flow from children to their parents in a parse tree.

Synthesized Attributes II

For example, given the following production:

$$\langle expression \rangle_1 \rightarrow \langle expression \rangle_2 + \langle term \rangle$$

We could add the following evaluation rule:

$$\langle expression \rangle_1.value = \langle expression \rangle_2.value + \langle term \rangle.value$$

S-Attributed Grammars

If we have an attribute grammar which **only** consists of **synthesized** attributes, we would call it an **S-Attributed Grammar**.

S-Attributed Grammars II

For instance, in our grammar of arithmetic expressions, we can assume that every integer literal has a *value* attribute which is equal to the parsed value of the its lexeme.

S-Attributed Grammars III

We can also assume that every non-terminal has a *value* attribute which is computed by either applying the operator from its right hand side to the *value* attributes of its other children or by simply propagating a *value* directly from a child if there is no operator.

Inherited Attributes

Inherited attributes are attributes evaluated using attributes from either a parent or a sibling.

Inherited Attributes II

For example, given the following production:

$$\langle procedure \rangle \rightarrow \langle definitions \rangle \langle statements \rangle$$

We could add the evaluation rule:

$$\langle statements \rangle.def = \langle procedure \rangle.def \cup \langle definitions \rangle.def$$

Inherited Attributes III

This would allow us to compute all of the definitions that are visible in a block of statements.

This data must flow down through the grammar, because the definitions must precede their uses.

L-Attributed Grammars

Another special type of attribute grammar is **L-Attributed Grammars** which are grammars whose attributes are either *synthesized* or *inherited from a parent or previous sibling*.

L-Attributed Grammars II

For example, given the following production:

$$A \rightarrow BCD$$

All inherited attributes for C must be computed from attributes decorating A or B .

L-Attributed Grammars III

Due to the nature of L-attributed grammars, they can be parsed and have all attributes evaluated in a single pass!

This is very convenient for building efficient language processors.

Attribute Grammars III

More generally, attribute grammars provide us with a way to capture many of the **static semantics** of a language which are needed for language processors.

Language Processors

Language Processing

Defining a language doesn't serve much purpose unless we have an efficient way to process the language for some purpose.

Language Processing II

Often, we will be either *compiling* or *interpreting* our language. However, we may also want to use similar techniques for creating *pretty printers*, *linters*, and other utilities.

Language Processors III

The creation of such programs is often intimidating, but, fortunately, they can be broken down into various independent steps which are well understood after decades of research and development.

(as long as we're not attempting to create cutting edge tools!)

Compiler Structure

When implementing a compiler, we will typically see the following stages:

- Lexical Analysis
- Syntactic Analysis
- Semantic Analysis
- Intermediate Representation Generation
- Optimization Passes

- Code Generation

Lexical Analysis

First, we will scan the characters of the input program and attempt to generate equivalent tokens from the alphabet of our language.

Syntactic Analysis

Once we have a string of tokens, we can then begin to perform the *syntax analysis* which will validate that the program structure matches the language grammar.

This is also known as: **Parsing**

Semantic Analysis

Once we have parsed the program, we will use the resulting *parse tree* to perform further analyses to ensure that the static semantics of the program are correct.

Intermediate Representation

At this point, we are certain that the program was valid and have a *parse tree* representing it.

Now we can, **optionally**, convert the parse tree to an *intermediate representation* which makes further steps simpler.

Optimizations

In order to improve the performance of the program, we can now analyse the intermediate representation and attempt to find “better,” semantically equivalent programs.

Code Generation

Once we have made the program as efficient as possible, we will walk the intermediate representation and convert it into native machine code.

Interpreter Structure

An interpreter will likely have a similar structure, though optimization and code generation will not be necessary.

- Lexical Analysis
- Syntactic Analysis
- Semantic Analysis
- Intermediate Representation Generation
- Interpretation of Intermediate Representation

Language Processors

We can group *compilers*, *interpreters*, and *transpilers* under the general category of *language translators*.

Going further, we can group translators, code analyzers, and various other types of programs under the more generic umbrella of *language processors*.

Language Processor Structure

If we look at the high level structure of a *language processor*, we'll find that we are much less certain about the details, but we can be pretty sure that, at the very least, *lexical* and *syntactic* analysis will be necessary.

Language Processor Structure II

Intermediate representation generation will also be common, but we'll see that we can often skip this for simple processors and rely on the *parse tree* as the intermediate representation.

Regular Expressions

Languages Again

We previously discussed the concept of *context free languages* which we use commonly use to represent programming languages.

There are, however, less complex classes of languages.

Regular Languages

Languages for things like variable identifiers, integer literals, and comments that we find in code fall into the class of *regular languages*.

Regular Grammars

Instead of a *context free grammar*, we can represent *regular languages* using *regular grammars*.

While these grammars are strictly less powerful than *context free grammars*, they have the benefit of being much easier to understand and implement.

Regular Grammars II

The general structure of a regular grammar is the same as a context free grammar, with the following constraints on the right hand side of productions:

- they can have *at most* one non-terminal symbol
- the non-terminal must be the first or last symbol

Regular Grammars III

You may remember the following production from our context free grammar discussion:

$$\langle intlit \rangle \rightarrow \langle digit \rangle | \langle intlit \rangle$$

and mentioned that it *could* be part of our syntax definition.

Regular Grammars IV

This doesn't *quite* satisfy our constraints, as it has *two* non-terminals on the right hand side.

However, the following does:

$$\langle intlit \rangle \rightarrow (0|1|2|3|4|5|6|7|8|9)\langle intlit \rangle$$

Regular Grammars V

Using this production, we can derive the integer 100 as follows:

$$\begin{aligned}\langle intlit \rangle &\rightarrow 1\langle intlit \rangle \\ &\rightarrow 10\langle intlit \rangle \\ &\rightarrow 100\end{aligned}$$

Regular Expressions

The definition of a regular expression consists of the following:

- an alphabet
- the operations
 - concatenation
 - alternation
 - Kleene star

Regular Expressions II

Recursively, we can define a regular expression over the alphabet Σ as follows:

First, the empty set $\emptyset$, the empty string ϵ , and any literal symbol $a \in \Sigma$ (denoting $\{a\}$) are valid regular expressions.

Concatenation

Given any two regular expressions R and S , the *concatenation* RS , representing the combination of strings in R combined with strings in S , is also a valid regular expression.

Alternation

Given any two regular expressions R and S , the union $(R|S)$ of strings in R and S is also a valid regular expression.

Kleene Star

Given a regular expression R , the Kleene closure R^* representing the repeated concatenation of R with itself is also a valid regular expression.

Regular Expression III

So, for example, the regular expression

$$(ab|ac)^* d (x|\epsilon)$$

Would define the language

$$\{d, dx, abd, abdx, acd, acdx, ababd, ababdx, \dots\}$$

Additional Notation

The more common notation you will see for regular expressions is to use the following operations:

- ? to indicate an optional expression
- + to indicate one or more repetitions
- ranges like [0..9]

Lexical Analysis

Lexical Analysis

As mentioned before, *lexical analysis* is responsible for transforming a sequence of characters into a sequence of *tokens*.

Lexemes and Tokens

In this context, a *token* is essentially a terminal symbol in our *context free grammar* which is *associated* with a sequence of characters, called a *lexeme*, in the original program.

Lexemes and Tokens II

For example, if we encounter the characters

```
final int x = 100
```

we might identify the following lexemes

$$final, int, x, =, 100$$

Regular Expressions

Note that the *way* we would identify these lexemes is by applying regular expressions to the input program.

Basically, every *token* we are interested in will have a regular expression associated with it.

Lexemes and Tokens III

In the process of identifying the lexemes in the input language, we can also determine the corresponding *token*.

final	int	x	=	100
final	id	id	equals	intliteral

Syntactical Analysis

We would also expect to have a production in our *grammar* like

$$\langle def \rangle \rightarrow [final] \langle id \rangle \langle id \rangle = \langle intliteral \rangle$$

Remember, when validating *syntax*, we don't care about the lexemes!

Syntactical Analysis II

The *lexer* confirms that the *lexemes* are valid and the *parser* confirms that the corresponding structure of the *tokens* is valid.

Semantic Analysis

Note, also, that we use the *id* token for *both* type and variable names in this example.

Wouldn't that be confusing for the *parser*?

Semantic Analysis II

No! The structure of the program does not need to distinguish between the two, because the *parser* doesn't know what types actually exist! It simply needs to know that valid *lexemes* for identifiers were seen.

Semantic Analysis III

Determining whether the first identifier is an *actual* type visible in the current scope is the responsibility of the *semantic analysis* phase.

Semantic Analysis IV

Until then, type and variable names are determined by the same regular expression, so they are interchangeable.

Tokens

Up to this point, we have talked about tokens very abstractly as “symbols” in our grammar.

We have also mentioned that the parser doesn’t have to know the underlying lexemes for a token.

Tokens II

In practice, though, later stages in our language processors will need this information, so our tokens will actually be *data structures* which store the underlying lexemes, the line/column of the lexeme, and other metadata.

Lexical Analysis II

Additionally, the lexer will also need to do things like eliminate white space and comments from the program, as they contribute nothing to the actual semantics.

Parsing

Parsing

As mentioned previously, once we convert our input to a string of **Tokens**, we can begin to **parse** it.

Parsing II

You can think of parsing as, roughly, the idea of taking a *sentence* from a language and trying to determine the steps which were taken to derive that sentence using a given grammar.

Parse Tree

During the process of discovering the correct derivation of the sentence, we can also construct a *parse tree* which is a structure that represents that derivation.

Parse Tree II

Once we have a parse tree, we can then evaluate it in an interpreter, rearrange the tree to optimize the structure, and/or use it as a guide for generating code.

Recursive Descent Parser

The simplest implementation of a parser for a given grammar would be a **recursive descent** parser.

Recursive Descent Parser II

A recursive descent parser is implemented by creating a method for *every* non-terminal symbol in the grammar which will do the following:

- Figure out what production to apply next
- Call all non-terminal methods on the rhs of the production
- Confirm all terminals along the way appear as expected

Example

Let's write a recursive descent parser for the following grammar:

$$\begin{aligned}\langle list \rangle &\rightarrow [\langle values \rangle] \\ \langle values \rangle &\rightarrow \langle value \rangle \\ &\quad | \langle value \rangle, \langle values \rangle \\ \langle value \rangle &\rightarrow \langle name \rangle \\ &\quad | \langle list \rangle\end{aligned}$$

Example II

First, we add the following method:

```
public void list(){
    match(lexer, Token.LBRACKET);
    values();
    match(lexer, Token.RBRACKET);
}
```

Example III

Next, we handle the $\$ \langle \text{value} \rangle \text{\$}$:

```
public void values(){
    value();
    if(lexer.peek()==Token.COMMA){
        match(lexer, Token.COMMA);
        values(); // Is this ok?
    }else{
        return;
    }
}
```

Example IV

Now we need to handle the $\langle \text{value} \rangle$ productions:

```
public void value(){
    if(lexer.peek()==Token.LBRACKET){
        list();
    }else{
        name();
    }
}
```

Example V

Since we're not generating a parse tree or evaluating the expression, we don't do much with the $\langle \text{name} \rangle$:

```
public void name(){
    match(lexer, Token.NAME);
}
```

Example VI

Note that all we're really doing here is *recognizing* sentences in the language we have defined.

The same general process, though, can be used to build a parse tree and, in some cases, *evaluate* sentences in the language.

Recursive Descent Parser III

Unfortunately, recursive descent parsers can not parse sentences from *every* context free grammar

They can, however, handle many interesting languages if we massage them a little bit.

Left Recursion

A major limitation of recursive descent parsers is that they can't parse **left recursive grammars**. In other words, the following would not be parsable.

$$\langle a \rangle \rightarrow \langle a \rangle + \langle b \rangle | \langle b \rangle$$

Why?

Left Recursion II

There are algorithms for refactoring such a grammar to *not* be left recursive, but we can sort of intuitively figure it out by thinking about what the production does:

It creates a series of $\langle b \rangle$ nonterminals separated by '+' operators.

Left Recursion III

This could also be done with the following productions:

$$\langle a \rangle \rightarrow \langle b \rangle | \langle b \rangle + \langle a \rangle$$

...but do they have the same parse tree?

Right Recursion

Isn't the new grammar **right recursive**?

Wouldn't that have the same problem as a **left recursive** grammar?

Right Recursion II

No, because, by the time we reach the recursion in a given production, we have already processed at least one terminal. This guarantees that we have made progress through the input between each recursive call.

Parse Trees

Parse Tree

We have seen how to build a language parser for the purpose of *recognizing* valid programs, but we want to do something more useful: **Create an in memory representation of our program.**

Parse Tree II

If we want to generate a parse tree with a recursive descent parser, we can do so by adding a return type to each method representing a tree node.

Parse Tree III

First, we handle the $\langle list \rangle$:

```
public Node list(){
    match(lexer, Token.LRACKET);
    Node result = new ListNode(values());
    match(lexer, Token.RBRACKET);
    return result;
}
```

Parse Tree IV

Next, we generate nodes for $\langle values \rangle$:

```

public List<Node> values(){
    final List<Node> nodes = new ArrayList<>();
    nodes.add(value());
    if(lexer.peek()==Token.COMMA){
        match(lexer, Token.COMMA);
        nodes.addAll(values());
    }
    return nodes;
}

```

Parse Tree V

And $\langle value \rangle$:

```

public Node value(){
    if(lexer.peek()==Token.LBRACKET){
        return list();
    }else{
        return name();
    }
}

```

Parse Tree VI

And now, $\langle name \rangle$ is a bit more interesting:

```

public Node name(){
    Token t = lexer.peek();
    match(lexer, Token.NAME);
    return new NameNode(t.getLexeme());
}

```

Remember: *Tokens* are data structures!

Parse Tree VII

This, actually, won't generate the parse tree exactly, since we have ignored some intermediate nodes that aren't necessary for representing the parsed data. That's ok, though.

Attribute Grammars, Again

We previously described the idea of *synthesized* attributes which can be computed in an *attribute grammar* with data flowing from child node to parent

nodes.

Recursive Descent

When writing a recursive descent parser, we *descend* down to the terminal symbols in our grammar, then we work our way back up the tree (i.e., a postorder traversal).

Synthesized Attributes

Accordingly, we would pass synthesized attributes up to parent nodes via return values.

Parse Tree VIII

In other words, we can view the creation of our parse tree as an example of parsing an attribute grammar where the synthesized attributes are in memory representations of the symbols and structure in our language grammar!

Synthesized Attributes II

For example, we could have defined the following:

$$\begin{aligned}\langle list \rangle &\rightarrow [\langle values \rangle] \\ \langle list \rangle_{node} &= ListNode(\langle values \rangle_{nodes}) \\ \langle values \rangle &\rightarrow \langle value \rangle \\ \langle values \rangle_{nodes} &= List(\langle value \rangle, node) \\ \langle values \rangle &\rightarrow \langle value \rangle, \langle values \rangle \\ \langle values \rangle_{nodes} &= \langle value \rangle_{node} :: \langle values \rangle_{nodes}\end{aligned}$$

Parse Tree IX

We typically would not do this, because the structure we are defining is simply the structure of the grammar itself (roughly), but the point is that this is a good example of an S-attributed grammar.

Inherited Attributes

We also discussed inherited attributes which are passed from parents *down* to their children or from *siblings* to other *siblings*.

Inherited Attributes II

In general, a grammar with inherited attributes can not be parsed in a single pass with a recursive descent parser, because we have to visit children before the subsequent siblings have been processed.

L-Attributed Grammars

However, at the time we visit a child, in a recursive descent parser, we have already processed all of its *previous siblings*.

This means that any L-Attributed grammar can be parsed by a recursive descent parser.

Inherited Attributes III

In our recursive descent parser, we can incorporate inherited attributes by *adding parameters to our methods*.

Attribute Grammars

In other words, in the context of a recursive descent parser, *return values* represent *synthesized attributes* and *parameters* represent *inherited attributes*.

Recursive Descent Parsing

This is the beauty of recursive descent parsing. The structure of the parser is roughly one to one with the grammar productions, and implementing attribute computations also follows directly.

Memory

Memory

We often talk, colloquially, about things being stored “in memory”, but we really need to be a bit more precise.

This requires that we have a bit deeper understanding of how programs use memory.

Von Neumann, Again

Remember, when we talked about how computers work, we mentioned that both data and code are stored in memory.

What we didn’t really discuss, was *how* they are stored in memory.

What is a program?

When loaded into memory, a program typically consists of:

- Text Segment
- Data Segments
- Heap
- Stack

Text Segment

The *Text Segment* is the part of the data stored in memory which actually stores the instructions to be executed.

Data Segments

Next, we will have *initialized* and *uninitialized* data segments which contain space (and values) allocated at the time the program is loaded into memory.

Heap and Stack

Finally, we have space available to be allocated *during the execution of our program* and contains heap and stack data.

There is no clear division between these two segments of memory, we simply put them on opposite ends and hope they don’t run into each other!

Heap and Stack II

The *Heap* is used for allocating memory in an ad hoc manner as needed by our code.

For instance, if we go through a loop and create a bunch of objects in Java, they will be stored in the heap.

The *Stack*, is used for allocating memory when the usage patterns work like a stack.

For instance, when we call a method, we know we can add the parameters to the stack, then “pop” them off when the method returns.

Addresses

To read and write values from and to memory, we need to have the *address* of the beginning of the block of memory where we are storing the data.

Addresses II

The segment in which a piece of data will live is determined, roughly, by *how and when we can compute its address*.

Paging

Note, however, that the addresses used by a program are actually *virtual addresses* and not the actual *physical* location of a value in memory, because the data may be *paged* in and out of memory at different locations at runtime.

Compilation

Compiling to an executable is, roughly, generating the text segment, data segments, and other metadata needed by the operating system to properly link and load the executable into memory.

Loading

Accordingly, loading a program is, roughly, just loading these segments into memory and setting the instruction pointer to the beginning of the text segment.

Warning

This is all a very rough simplification, but it's a good enough model to use for this course.

Addresses And Lifetimes

Bindings

Generally speaking, a **binding** is an association between an attribute and an entity.

For example, a *variable* might be **bound** to a *value* or to a *type*.

Binding Time

We will say that a binding takes place at its **binding time**. The binding times of a given programming language can have a huge impact on its semantics.

Binding Time II

For example, a *variable* in Java is bound to its *type* at **compile time**.

It may not be bound to its *value*, though, until *run* time.

Binding Time III

Some aspects of a language, like the internal representation of its values may be bound at **compiler design time**.

For example, how would a **struct** be stored in memory?

Binding Time IV

Generally speaking, we have the following binding times:

- Language design time
- Language implementation time
- Compile time
- Load time
- Link time
- Run time

Bindings II

If a binding occurs before execution time and *can not be changed*, we say it is **static**.

Otherwise, it is a **dynamic** binding.

Variables

Variables can be described as a sextuple of attributes:

- Name
- Address
- Value
- Type
- Lifetime
- Scope

Names

Names are boring.

Addresses

Addresses are considerably more interesting!

At a high level, this is simply the location in memory which a variable is bound to.

The details are complicated, though!

Storage Bindings

In order to store a value, memory must be **allocated** from the available, free memory.

The **lifetime** of a storage binding extends from the time a variable is bound to a memory location until the time it is unbound.

Scope

Warning: Do not confuse this with the **scope** of a variable, which we will discuss later.

Static Variables

A **static** variable can be bound to its storage at compile time. In other words, the location to which the variable will be bound is determined before execution and will not change.

Data Segments

In other words, since we can choose an address at compile time and that address won't change, we can include the data for that variable in the *data segment* of our memory layout.

Static Variables II

Since the binding is static, all references to the underlying memory can be *direct*.

There is also no cost for allocation, as the memory required is known and “allocated” as part of the loading process for an executable.

Static Variables III

However, it prevents things like recursive subprograms and it can be wasteful since the memory does not get deallocated.

Stack-Dynamic Variables

Stack-dynamic variables are bound to storage when their declarations are **elaborated**.

In other words, every time the code where their declarations are located is executed, storage is allocated and bound.

Stack-Dynamic Variables II

A typical example of such variables are local variables in a sub-program.

Every time the sub-program is called, these variables are bound to newly allocated storage (and deallocated on subprogram exit).

Stack-Dynamic Variables III

The allocation of these variables is on the stack and, accordingly, the addresses are only known at run time (because we don't know the order in which subprograms will be called at compile time!)

Stack-Dynamic Variables IV

Since we don't know the addresses at compile time, *indirection* is required to access the memory at run time.

Typically, this will be done by computing an offset from the stack pointer.

Explicit Heap-Dynamic Variables

At compile time we may not know the exact memory location where a variable can be stored *or* where the variables are located in reference to the stack pointer.

Explicit H-D Variables II

For instance, if we want to create a linked list in a function, we can allocate a pointer on the stack, but we don't know how much space we need to allocate for the list, since it will be built dynamically.

Explicit H-D Variables III

Instead, we must allocate from the *heap*, which has no particular order to its contents.

So, the pointer will be on the stack, but it will point to some arbitrary location in the heap.

Explicit H-D Variables IV

```
Node *head = new Node();
while(/* some condition*/){
    head.next = new Node();
}
```

Explicit H-D Variables V

These variables are *explicit* heap-dynamic variable, because they must explicitly be allocated with an operator (like **new**) or a subprogram (like **malloc**).

Explicit H-D Variables VI

Due to the unorganized nature of the heap, using heap-dynamic variables can be expensive, because you have to load the address, then use it to load the data itself.

Explicit H-D Variable VII

If the memory is not *deallocated* properly, you will also end up with memory leaks!

Java became famous largely because its explicit heap dynamic variables are *automatically* deallocated by using a *garbage collection* algorithm.

Implicit Heap-Dynamic Variables

In languages with *implicit* heap dynamic variables, *allocation* as well as *deallocation* are performed automatically.

Implicit H-D Variables II

Often, this approach is seen in dynamic languages which do not bind values, types, or storage location until run time.

This can lead to a large overhead for the extra book-keeping required in such languages.

Scope And Types

Scope

In addition to a *lifetime* and *address*, a variable will be bound to a *scope* which, for any given location in the code, determines what variables may be referenced.

Scope II

Note: scope and lifetime may not be the same!

Static Scope

Most languages use what is referred to as “static” scope or “lexical” scope.

Using this approach, the variables that are *visible* at a given location are determined *statically* by the *structure of the code*.

Static Scope II

Given a name in a particular statement, we can determine which variable is being referred to (if such a variable exists!) by looking at the *local* declarations, then declarations in the *static parent*, then declarations in the *static ancestors* of the block where the reference was made.

Global Scope

Variables defined outside of any subprogram may be visible *globally*.

In dynamic languages, this can be somewhat confusing!

Dynamic Scope

Another approach that was somewhat popular in the past before people realized how complicated it can make things was *dynamic scope*.

Dynamic Scope II

Using this approach, visibility is determined by the *order in which subprograms are called*!

Type Binding

Since all data in a modern computer is stored as, essentially, binary numbers, we **must** bind a variable to a *type* before it can be used.

Static Type Binding

In languages which use *static* typing, the type of every variable must be determined at compile time.

Static Type Binding II

These definitions can be either *explicit* or *implicit*.

Either way, all types must be known before execution. This has the benefit of enabling the compiler to detect many errors at compile time!

Dynamic Type Binding

Alternatively, a variable's type may not be bound until it is bound to a variable.

These bindings may also change!

Dynamic Type Binding II

This gives the programmer a lot of flexibility, but also prevents the detection of common errors, because of the lack of information available at compile time.

Strong vs Weak Types

A related concept we will discuss later is whether a variable's type binding is **strong** or **weak**.

Roughly, whether or not the binding *can change*.

Type Systems

This give us a two dimensional classification for type systems in a given language:

- *static* or *dynamic*
- *strong* or *weak*

Primitive Types

Types

As we mentioned earlier in the semester, all data is, ultimately, just patterns of bits in memory.

Roughly, it's all just numbers.

Types II

Types, basically, are how we explain to the compiler how we want our data to be interpreted.

This includes specifying the possible values and operations which can be applied to those values.

Primitive Data Types

While most **data types** are defined in terms of *other* data types, the **primitive data types** are the essential types provided by a language.

Primitive Data Types II

In many cases, primitive data types are the data types which the underlying hardware can support directly.

Integers

For instance, a processor can almost always “understand” integers and a certain set of operators which can be used with integers.

Integers II

In Java, we have **byte**, **short**, **int** and **long** types which are 8, 16, 32 and 64 bits in length, respectively.

Integers III

These sizes correspond to different multiples of a byte, but also match the sizes of memory which most common processors can handle directly.

Integers IV

Note, though, that the Java spec requires the precise sizes *whether they are directly supported by the underlying architecture or not!*

Integers V

C++, on the other hand, states *minimum* lengths for each integer type, but leaves the exact size as an implementation detail which can vary across compilers and architectures.

Why though?

Why might you choose one of these approaches over the other?

Integers VI

In the context of integers, a language designer must also decide whether integers are **signed**, **unsigned**, or potentially both.

Floating Point

Floating point types allow us to store values, roughly, like real numbers. Again, though, the most common floating point standard (IEEE 754), is usually supported directly in hardware.

Floating Point II

This standard defines 32 bit *floats* and 64 bit *doubles* which represent the commonly supported floating point types in programming languages.

Floating Point III

Floating point values are stored in a format which is more or less like scientific notation. There is a sign bit, an exponent and a fraction.

Floating Point IV

Floating point values, though, are not continuous like the real numbers!

For example, you can not represent the value 0.1 precisely in binary.

Decimals

In many business applications, though, it is necessary to ensure that all values are stored with exact precision. To support this, many languages (and some hardware) support primitive **decimal** types.

Decimals II

These types can be thought of, roughly, as storing a series of *digits*. A naive implementation, though, is very wasteful, because hardware usually can not access data smaller than a byte (but a digit only requires four bits).

Decimals III

In practice, the decimal values are stored as **binary coded decimals** which pack two digits in a byte. This must be supported by hardware to be efficient, and is still wasteful.

Decimals IV

Such hardware support *seems* to be less common and decimal types are often implemented in software, even if they are presented as primitive types.

Decimals V

The decimal type demonstrates, again, a fundamental issue with respect to getting computers to do what we want: computers can only efficiently process a very limited collection of types!

Decimals VI

Even something as conceptually simple as accurate currency values may require software support which makes them significantly less efficient than integers.

Booleans

Boolean types consist, simply, of the values *true* and *false*. Typically, these values are represented by a single bit, though again, this is wasteful, since most hardware can not address a single bit directly.

Booleans II

In hardware, conditionals using booleans are often computed as “zero or not.” In other words, any value that evaluates to 0 is false, and any other value is true.

Booleans III

Booleans can be efficiently *processed* in hardware, but again we see that they don’t quite fit into the paradigm of byte addressable architecture. Computers really are limited!

Characters

Characters are another example of somewhat ill fitting, yet conceptually simple, types. Characters are typically not supported in hardware directly.

Why?

Encodings

Basically, characters are *encoded* using some sort of mapping between each character and an integer (or a few).

It’s then the responsibility of software to interpret these corresponding integers as characters.

Encodings II

However, there are *several* character encodings which are in use, which means a given integer could represent several different characters, depending on the encoding being used.

Unicode

Standardization efforts have been undertaken to make this less of an issue by specifying a universal character set referred to as “Unicode.” There are, however, multiple unicode encodings...

Characters II

An important takeaway, though, is that characters are a conceptually simple type that has very little direct support in the underlying hardware, though they are still typically considered to be “primitive”.

Enumerations

Enumerations are, usually, syntactic sugar for safe integer constants representing enumerations of values.

(Though, not always. See Java)

Enumerations II

They can be dangerous, though, if they are not truly treated as a unique *type* and are instead treated as simple integers!

Strings and Aggregates

Strings

We are all familiar with **string** types. Strings, though, are complicated enough to require a bit of discussion from the language design and implementation perspective.

Strings II

One question is: are strings a special type, or are they merely arrays or lists of characters? In other words, are they truly “primitive” or not?

Strings III

Another interesting question is whether or not strings should have dynamic lengths.

Static Length Strings

Static length strings have their lengths determined when they are created, which can not be changed.

Such strings are often *immutable*

Limited Dynamic Length Strings

“C style” **limited dynamic length** strings are arrays of characters with a special *null* value which indicates the end of the string.

Limited Dynamic Length Strings II

A certain amount of space is allocated for a string which determines its *maximum* length.

The *null* terminator indicates the actual length.

Working with such strings can be very dangerous!

Dynamic Length Strings

Dynamic length strings have unbounded, variable lengths. Such strings are flexible, but can be expensive.

Dynamic String Implementation

Dynamic length strings can be stored as linked lists, which are easily extended, but this introduces a lot of overhead since it will be on the heap.

Dynamic String Implementation II

Alternatively, they can be stored in adjacent bytes, but expanding the size typically requires copying all of the data!

Arrays

Arrays are homogeneous aggregates of data elements.

(though they may not always *seem* homogeneous)

Arrays II

Arrays, typically, correspond to adjacent bytes in memory, though they may reference values elsewhere on the heap.

Indices

Arrays can be *indexed* to allow access to a given element.

Perhaps surprisingly, the indices are not always integers!

Indices II

```
type Character_Map is array (Character range 'A' .. 'Z') of Natural;
```

Static Arrays

A static array has its index range and its size set at compile time. It is also statically bound to its allocation. This is very efficient, but not very flexible!

Fixed Stack Dynamic Array

A fixed, stack dynamic array has its size set at compile time, but its (de)allocation occurs at runtime on the stack.

Fixed Heap Dynamic Arrays

A fixed, heap dynamic array has its size and allocation bound at runtime on demand.

Heap Dynamic Arrays

Heap dynamic arrays can have their size and storage allocation modified at runtime.

Multiple Dimensions

In some languages **rectangular arrays** are supported, while others only support **jagged arrays** (basically, arrays of arrays).

Array Slicing

A neat feature in some languages like Python is array slicing, which allows you to index sub-arrays:

```
ys = xs[3:6]
```

Records

A **record** is an aggregate, heterogenous data type which is stored in *adjacent memory locations*.

Records II

Records provide the ability to access their **fields** *by name*.

Records III

Field lookups are efficient, because the fields are at constant offsets in memory. In other words, no complex indexing computation is necessary.

Tuples

A **tuple** is similar to a record, except for the fact that it does not have named fields.

Instead, the elements of a tuple are accessed by *position* or by pattern matching.

Lists

Lists are similar in behavior to arrays, but tend to have slightly different semantics and implementations.

For instance, lists may not provide direct support for indexing, if they are implemented as linked lists.

Lists II

Lists will often provide operations for appending an element to the list, for accessing the first element, and for accessing the rest of the list (other than the first element).

Unions

A **union** is a type which can store a *single* value of different types at different points of execution.

For instance, a union could be a float, then later be an int!

Unions II

```
union flexType {
    int intEl;
    float floatEl;
};

union flexType el1;

el1.intEl = 27;
x = el1.floatEl;

Isn't this just a record?
```

Unions III

The storage for this type does *not* contain an int and a float. Enough storage is allocated to store either an int or a float *in the same place*.

The above code writes an int to the allocated memory, then reads it as a float!

Unions IV

Such **free unions** are dangerous.

Discriminated unions, on the other hand, allow discriminants to be used to make the unions safe.

For example, sum types in Haskell are discriminated unions.

Reference Types

Reference Types

Up to this point we have been discussing **value types**. Now we will turn to a discussion of **reference types** which are not themselves structured and merely refer to other things.

Pointers

The most common reference type is a **pointer**. Pointers can allow for indirectly referencing values and for referring to heap dynamic variables.

Heap

Remember that the heap is an unorganized portion of memory where we can *allocate* memory *dynamically* to store values.

Heap II

Since these values are not at a static location or at a statically determined stack offset, we can not directly refer to them.

Pointers II

Instead, we refer to a *pointer* which will store the dynamically determined address of the value.

Pointers III

When we want to access the value itself, we must **dereference** the pointer. This takes the address stored in the pointer and uses it to locate the value.

Pointers IV

```
int x = 4;    // Stack dynamic!
int* px = &x; // Store the address of x
int y = *px;  // Dereference px
```

Pointers V

```
struct Node {  
    int val;  
    Node* next;  
};  
  
// ...  
  
Node* n = new Node();  
(*n).val = 10;  
(*n).next = NULL;
```

Pointers VI

Alternatively...

```
Node * n = new Node();  
n->val = 10;  
n->next = NULL;
```

Pointers VII

Alternatively...

```
Node *n = (Node*) malloc(sizeof(Node));  
n->val = 10;  
n->next = NULL;
```

Dangling Pointers

A common error is the creation of a **dangling pointer**, which refers to an address where there is no longer an allocated value.

Memory Leaks

Another common problem is allowing a pointer to go out of scope or reassigning a pointer and “losing” the address to a value on the heap. Once we lose the address, we can’t deallocate!

Pointer Arithmetic

In some languages, we can perform arithmetic on a pointer to, for example, compute a new address to the “next” value.

Danger!

Clearly, pointers are powerful, **but dangerous!**

Accordingly, many modern languages do not include pointers.

References

Instead, they provide **references** which are *not* references to an address, but are, rather, a reference to an object in memory.

References II

Basically, you can think of a reference as a pointer which is implicitly dereferenced and never exposes an address.

References III

We still have to worry about memory leaks and dangling references, though.

Accordingly, many modern languages will also provide **garbage collection**.

Heap Management

Heap Management

We have, at a very high level, discussed how the heap is used and hinted at how it is managed.

Now we want to go into a bit more detail about how memory on the heap is allocated, used and deallocated.

malloc

Previously, we saw that we could use `malloc` to allocate a certain number of bytes on the heap.

There is a lot of hand waving involved in this description, though.

Data Segment

First, we sort of glossed over how the heap actually gets allocated to begin with. Conceptually, we can think of it as using whatever empty space is left by the stack.

In practice, though, the heap will be allocated as part of the *data segment*!

Data Segment II

This may seem odd, since we don't know what the actual memory accesses will be, and the data segments are calculated statically.

Data Segment III

In reality, some amount of heap will be allocated statically on program load, but calls to `malloc` may actually have the OS extend the data segment so that more heap space will be available.

Data Segment IV

In other words, there is an ever increasing boundary from the heap side similar to the stack!

This boundary helps to protect against errors like reading memory that should not be read and allows us to easily determine when the stack and heap space collide.

Fragmentation

However, unlike the stack, the heap space does not grow and shrink in an orderly manner. The *maximum* heap space is determined by the boundary, but within the bounds of the heap, there could be many different algorithms for actually choosing where to allocate memory for a given value.

Fragmentation

The goal of such algorithms is to try and ensure that the heap space is used efficiently instead of creating many small, useless gaps in the memory space.

Pointers Are Hard

We previously discussed pointers and how they can be difficult to work with directly, because of the possibility of memory leaks and dangling pointers which will lead to **undetected**, erroneous behavior.

Tombstones

One attempt to make pointers safer was the use of “tombstones” which would, basically, create an extra pointer for each heap dynamic value which would always exist in memory.

Tombstones II

If a value is deallocated, the tombstone pointer would be changed to `NULL` and any attempt to continue using the pointer would throw an exception.

This is very inefficient in time and space!

Locks And Keys

Another attempt was to add an additional integer value to each pointer and heap dynamic value, such that the values must match in order for a pointer access to be valid.

Locks And Keys

On deallocation, a values lock would be set to an illegal value so that no key will match, and an error can be detected.

Garbage Collection

Even with these algorithms, it’s still clear that the real solution to the problem of working with pointers correctly is to simply remove the allocation and deallocation from the control of the programmer.

Garbage Collection II

The concept of “garbage collection” is to simply allow garbage to accumulate on the heap instead of manually deallocating, which ensures that there can be no dangling pointers.

Garbage Collection III

Of course, this would basically mean that we would have severe memory leaks. So, once we are *certain* there are no remaining pointers to some allocated value, we can deallocate it.

Garbage Collection IV

In other words, the crux of garbage collection algorithms is *how do you determine when a value is no longer referenced by any in scope pointers?*

Reference Counters

One approach is to use *reference counters* which are counters added to allocated memory cells which are incremented and decremented as pointers are manipulated.

If a counter ever reaches zero, the memory can be deallocated.

Mark And Sweep

Another approach is the “mark and sweep” algorithm which periodically does the following:

- Mark all allocated memory as “garbage”
- From each in scope pointer, trace all linked memory cells, recursively and mark all encountered cells as *live*
- Deallocate all cells still marked as garbage

Mark And Sweep II

Mark and sweep algorithms were, however, very inefficient, because they required the program execution to be paused while performing the necessary scans.

Incremental Execution

The reference counting algorithm has the benefit of working *incrementally* without requiring a pause in execution.

Incremental Execution II

More recent mark and sweep algorithms, though, have been implemented to support incremental execution and are now very efficient.

The Future?

The *Rust* language has been developed with the goal of c/c++ like performance with the memory safety of Java.

Ownership

Roughly, every value in Rust must have a variable as its “owner”. At any given point in time, there can only be *one* owner. As soon as the owner goes out of scope, we know that the value can be deallocated.

Ownership II

Of course, this is basically saying that we can never have more than one reference to a value, which is incredibly limiting!

Borrowing

To allow for sharing a reference to a variable, Rust permits “borrowing” references.

Rust’s borrowing rules ensure that borrowed references will always be valid.

Borrowing II

Such borrows are not as flexible as plain old pointers, but are flexible enough for most use cases while also guaranteeing garbage collection like ease of use without actually garbage collection!

Type Checking

Type Systems

A type system is a tractable **syntactic** method for **proving** the absence of certain program behaviors by classifying phrases according to the kinds of values they compute

Type Systems II

Alternatively, we might say a *type system* is a set of types and the rules that govern their use.

Type Systems III

Accordingly, type systems can be leveraged to:

- detect errors *early*
- enable safer code maintenance
- support abstraction
- provide documentation
- provide hints to the compiler for optimizations

Type Systems IV

Additionally, type systems are being applied to problems like security, code analysis, automated theorem proving, and more.

Type Checking

A related concept, *type checking* is concerned more narrowly with ensuring that only compatible types of values are used with a given operator, function or expression.

Type Checking II

In some languages, type checking is performed statically, according to the languages type system, perhaps with some additional runtime checks.

Type Checking III

In other languages, type checking consists solely of runtime checks.

Language Safety

Safe languages can be described as languages that protect their abstractions from corruption.

Language Safety II

For instance, if you create an array of elements, you should expect that the array will only be updated via array operations and not via random memory writes from other parts of your code.

Language Safety III

Accordingly, for writing correct, safe code, we would prefer to use a language with a rigorous, static type system with guaranteed safety.

Strong Typing

We say a language has *strong typing* when type errors are *always* detected.

Grey Areas

Of course, all of these definitions are going to be somewhat ambiguous.

For instance, Java is mostly strongly typed, but allows arbitrary casting which technically means it isn't.

Grey Areas II

Also, expressions in Java are strongly typed, but there are *implicit* conversions that may prevent logic errors from being detected.

Grey Areas III

For this reason, languages like Ada will require that all conversions be explicitly invoked by the programmer.

Which can be very tedious, as we'll see when we look at more Ada features later.

Type Equivalence

One important aspect of type checking is determining *type equivalence*.

Roughly, types are equivalent in a given type system if the two types are interchangeable with respect to some operations.

Name Type Equivalence

The easier, but less flexible type of equivalence is *name type equivalence*, which simply treats types as equivalent, as long as they have the same literal type name.

Structure Type Equivalence

Structure type equivalence, on the other hand, considers values to be of equivalent types *as long as they have the same structure*.

Type Equivalence II

```
type Derived_Small_Int is new Integer range 1..100;  
subtype Subrange_Small_Int is Integer range 1..100;
```

For example, both of these types are, in essence, integers in the same range, but Ada uses *name type equivalence* for the first and *structure type equivalence* for the second.

Type Equivalence III

So `Derived_Small_Int` is not compatible with `Integer`, but `Subrange_Small_Int` is!

Why Not Both?

In practice, there are costs and benefits to both approaches, so languages often mix and match.

Productivity

As a side note, people often choose languages that don't satisfy these criteria, because they (supposedly) are more productive.

Productivity II

This is usually not true, in practice, because the overwhelming majority of the cost for software development is *maintenance*, which benefits more from safety than being able to write code *quickly*.

Productivity III

The lack of support from a rigorous type system is also likely to result in *more* maintenance, as otherwise trivial to detect errors will be introduced which need to be fixed later.

Productivity IV

In other words, you can write *incorrect* code a lot faster with such languages, but it is likely to take longer to reach the *same level of quality* as with safer languages.

OOP

Object Oriented Programming

Now we're going to talk about some OOP specific design considerations based on some of the things we have discussed so far.

Inheritance

you are already familiar with the rough idea of inheritance. When one class extends another, it “inherits” variables and methods from that class.

Inheritance II

The subclass is then free to add *additional* variables and methods which it needs.

It can also, perhaps, *modify* methods in the parent by *overriding* them.

Inheritance III

Typically, though, an OOP language will provide the ability to restrict the extent to which variables and methods are inherited.

Multiple Inheritance

In some languages, it is possible to extend multiple parents. This can be problematic, as there can be collisions which require additional effort to clearly distinguish.

Single Inheritance

The more modern trend is to only allow extending a single parent, but allowing multiple interfaces or similar constructs to be used.

Why is this not a problem?

Instance Fields/Methods

Instance fields and methods belong to an *instance* of a class.

Each instance has a copy of each instance variable.

An instance is required to call an instance method.

Class Fields/Methods

Class fields and methods, instead, belong to the *class* itself.

There is only one copy of a class variable.

No instance is required to call a class method.

Method Binding

At run time, it's possible that we might have a reference of type **A** which could point to an instance of type **A** *or any subclass of A*. Any subclass could have also overridden a given method in **A**.

When we call a method using this reference, which copy of the method should be executed?

Dynamic Binding

If dynamic binding is used, then the method of the *actual instance* will be chosen instead of the method in type **A** (unless, of course, it is an instance of **A**).

Code must exist at runtime to perform this binding.

Static Binding

Alternatively, we could statically bind the call to the type of the reference *no matter what the class of the actual instance is*.

When languages allow both, this can get confusing and is error prone!

Purity

Another important design decision whether or not *everything* should be an object (as is the case in Ruby).

This is often *not* the case, because primitive types, for example are more efficient, so they are leveraged when possible.

Allocation on Stack

In theory, we can allocate objects on the stack, but if subclasses add fields, how do we know how much space needs to be allocated?

This could result in *object slicing* which truncates the data to fit in the allocated stack (with surprising results!).

Heap Allocation

Alternatively, we can allocate objects on the heap. This simplifies the above problem, but also introduces inefficiencies due to the indirection and brings up the question of how deallocation is handled.

Heap Allocation II

As mentioned before, we can require that the user manually deallocate the heap memory, or we can implement garbage collection to ensure that everything is automatically deallocated.

Initialization

When we are dealing with subclasses, we must also decide whether or not the programmer must also manually initialize its parent, or if this should be automatic.

Subprograms

Subprograms

A subprogram is a type of *process abstraction* which allows a programmer to reuse code and abstract away its details.

Subprograms II

We will use the term *subprogram* to refer to *functions*, *procedures* and *methods*, as it is a bit more general.

(Though I will likely forget to do this!)

Subprograms III

Typically, though, we refer to a subprogram which returns as a *function* and subprograms that do not as *procedures*.

(and, of course, there are *methods* in the context of a class)

Terminology

A *subprogram definition* consists of a *subprogram header* (also called a *signature*) and a *subprogram body*.

Terminology II

The *parameter profile* describes the number, order, and type of a subprograms *formal parameters*.

A *subprogram declaration* is essentially a subprogram definition, but without the body (also called a *prototype*).

Terminology III

When a subprogram is *called*, we supply the *name* of the subprogram and the **actual parameters** which are bound to the **formal parameters** in the subprogram definition.

Parameters

Aside from non-local, visible variables, a subprogram can only access data provided via parameter passing.

(Sort of? What about methods in object oriented languages?)

Parameters II

This functionality also give us the ability to “parameterize” a segment of code. This gives a significant increase in flexibility.

Parameters III

Details about the number of parameters, the ordering, default values, named parameters, etc. get rather interesting, but we wont talk about that in class.

See the book for more information!

Parameter Passing Methods

We do want to focus on **parameter passing methods** which describe the semantics of what it means to “pass” parameters to a subprogram in a given language.

In Mode

If a formal parameter can *receive* data from an actual parameter, then we say that it is an **in mode** parameter.

Out Mode

If a formal parameter can *transmit* data to an actual parameter, then we say it is an **out mode** parameter.

Inout Mode

Not surprisingly, an **inout mode** formal parameter can both receive and transmit data from and to the actual parameter.

Semantics vs Implementation

Pay attention to the fact that we haven't discussed the *implementation* of these parameter modes.

We have only discussed the *semantics* of how the parameters will behave.

Java

What mode does Java use?

Pass By Value

One implementation method which is often employed is **pass by value**, which literally copies parameters into a subprogram!

Pass By Value II

This can often lead to unexpected results:

```
void swap(int a, int b){
    int temporary = a;
    a = b;
    b = temporary;
}
```

Pass By Result

We can also implement *out mode* parameters by copying values *from* a subprogram *to* its caller.

No value is provided to the subprogram at all, but space must be allocated to store the result value.

Pass By Result II

This, too, can lead to unexpected behavior:

```
void Something(out int x, out int y){  
    x = 10;  
    y = 23;  
}
```

```
Something(out a, out a);
```

Pass By Result III

```
void Something(out int x, int index){  
    x = 17;  
    index = 42;  
}  
//...  
i = 21;  
Something(list[i], i);
```

Pass By Value Result

And, of course, we can combine the two by copying data in and out of the called subprogram.

This is sometimes referred to as **pass by copy**.

Without Copying

As an aside, we could come up with some scheme which allows the above to be implemented without actually copying values, but that is usually *not* done.

Pass By Reference

As an alternative, we can simply share the *address* of the actual parameter with the subprogram and *share* it, in essence.

Pass By Reference II

This can be more efficient than copying parameters, however, it also leads to slower accesses in general, because of the indirection it introduces.

Pass By Reference III

This approach does not, however, control access and can allow for aliasing, both of which make writing and understanding code more complex.

Constant References

In C++, you can use *constant* references which prevent modifications in order to get the benefits of pass by reference without the risks.

More Subprograms

Subprograms As Params

Some problems are elegantly solved by passing subprograms *as parameters* to other subprograms.

Subprograms As Params II

However, this can be confusing, because we have to consider which referencing environment should be used for determining the visibility of variables.

Shallow Binding

Similar to dynamic scoping, *shallow binding* treats the environment of the caller of the subprogram parameter as the referencing environment.

Example

```
function sub1() {
  var x;          // deep Binding
  function sub2() {
    print(x)
  };
  function sub3() {
    var x = 3; // ad hoc binding
    sub4(sub2);
  };
  function sub4(subx) {
    var x = 4; // shallow binding
    subx();
  };
  x = 1;
  sub3();
};
```

Deep Binding

Similar to lexical scoping, *deep binding* treats the environment where the subprogram parameter *was defined* as the referencing environment for the subprogram.

Ad Hoc Binding

When *ad hoc binding* is used, the referencing environment is *the environment where the subprogram was passed as a parameter!*

Closures

A **closure** is a subprogram *plus* its referencing environment.

Closures are interesting, and necessary, because its possible that a subprogram might be called when its lexical ancestors are *not* currently activated!

Closures II

```
public Function<String,String> getFunc(final String input){
  final String prefix = "hello: ";
  return s -> prefix + input + s;
}
```

```
public void something(){
    final Function<String,String> f = getFunc("Phillip ");
    final String result = f.apply("Wright");
}
```

Closures III

Note that static scoping rules still apply as usual! The catch is that the *lifetime* of variables may need to be extended.

This may also force heap allocation instead of stack dynamic allocation.

Subprogram Pointers

Instead of passing a subprogram as a value, a language might require that a subprogram *pointer* be passed.

```
float (*pfun)(float, int); // Go home c++, you're drunk
```

Side Effects

Another issue to consider when discussing subprograms is **side effects**.

Side effects can make code very difficult to understand, so many languages don't allow anything other than in mode parameters.

Side Effects II

However, you may still have side effects if you update non-local values.

So try not to do that, either!

Side Effects III

In pure functional languages, side effects are mostly impossible, since they don't have *variables*!

Side Effects IV

One problem with severely limiting side effects, though, is that it can make some solutions difficult to implement.

For instance, what if you need to return two values from a subprogram?

Return Values

In most languages, only a single value can be returned by a subprogram.

In most languages, to work around this, you must return a tuple, list, or some other data structure.

Return Values II

Though some languages provide direct support for returning multiple values!

```
a, b, c = fun()
```

Overloading

It's often convenient to provide multiple definitions of a subprogram which operate on different types of data (or a different number of parameters).

When these definitions all have the same name (but different signatures!), we call it overloading.

Overloaded Operators

An interesting, related idea is overloading operators, which is common in languages.

Less common is *allowing end users* to overload operations!

Polymorphic Subprograms

A **polymorphic** subprogram takes parameters of different types on different activations.

Ad Hoc Polymorphism

Overloaded subprograms are a type of **ad hoc polymorphism**, which may not behave in a similar manner.

Subtype Polymorphism

In object oriented languages we usually have **subtype polymorphism** which is derived from the fact that we can pass any *subtype* of T as a parameter when a value of type T is expected.

Parametric Polymorphism

A *parametric polymorphic* subprogram allows for the specification of type parameters which can be used when defining a subprogram.

The subprogram can then be instantiated with various arbitrary types.

Parametric Polymorphism II

Such subprograms are often referred to as *generic* subprograms.

Dynamic Lang Stuff

In dynamic languages where parameter types don't really exist, you can have a type of polymorphism in the sense that you can pass in anything you want and, as long as the operators can handle it, it will work.

(for some definition of “work”)

A Taste of Haskell

Some Odd Bits

For most people, there are a few aspects of the Haskell language which will seem quite odd.

Let's talk about those first.

Expressions vs Statements

An **expression** is, roughly, a statement which returns a value.

Haskell programs consist entirely of **expressions**.

If Expressions

For example, in Haskell, **if** constructs *must* return a *value* on each branch.

```
x = 3
s = if x `mod` 2 == 0 then "hello" else "bye"
```

No Side Effects

Every expression must return a value.

Every expression *can only* return a value.

Everything Is Immutable

You can bind names to a value, but can not update bindings (that would be a side effect!)

Evaluation Is Lazy

Values are not computed until they are needed.

So, for example, you can do things like define infinite structures and process them without any issues.

Everything Is A Value

Functions, for example, are treated just like integers, booleans, etc.

They can be passed as parameters, returned from functions, stored in data structures, and bound to names.

Time To Code

Now let's go ahead and start writing some code to help make these ideas clear.

Sum Types

Many types are simply a collection of possible values.

For example, a boolean is either **true** or **false** and natural numbers are 0, 1, 2,

Sum Types II

Such types (**sum types**) can be easily defined in Haskell:

```
data TrafficLight = Red | Yellow | Green deriving Show
light1 = Red
```

Pay attention to the type inference!

Product Types

Some types are defined as the cross product of *other* types.

For example, a three dimensional point is basically, three integers. Or, alternatively, $\mathbb{Z} \times \mathbb{Z} \times \mathbb{Z}$

Product Types II

Such types (**product types**) can also be easily defined in Haskell:

```
data Point = Point Int Int Int deriving Show
p = Point 10 20 12
```

Algebraic Data Types

More generally, we are using **algebraic data types**.

We can easily mix and match sum and product types:

```
data Point = OneD Int | TwoD Int Int | ThreeD Int Int Int
```

Functions

Another oddity...

All functions take one argument and return one result.

```
add x y = x + y
```

...but this compiles. Why?

Currying

While that *looks* like a function with two parameters, the type is actually:

$$Int \rightarrow Int \rightarrow Int$$

Currying

In other words, a function that takes an *Int* and return a function which takes *another Int* and returns the sum of both provided *Ints*.

Partial Application

This isn't just semantics!

```
addFour = add 4
```

By *partially applying* the add function, we have created a new function!

Pattern Matching

We can also use pattern matching and our algebraic data types to make implementing functions nice and clean:

```
data TruthValue = T | F
und T T = T
und _ _ = False
```

Lists

We can create lists:

```
xs = [Red, Red, Green]
ys = [(+), (-)]
zs = [1..]
```

Lists II

And we can work with them in interesting ways:

```
a = head zs
b = tail zs
c = head (tail zs)
d = drop 5 zs
e = take 5 zs
```

Lazy Evaluation II

Remember, evaluation is lazy, so it's not a problem to work with infinite lists!

Lists III

```
nats = [1..]
take 10 nats
```

...but that's kind of boring

Higher Order Functions

Things get more interesting when we start using *higher order functions*.

In other words, functions that take other functions as parameters or return a function.

Map

One example is the function **map**, which takes a function and a list*, and applies the function to every element in the list.

* This isn't exactly true, but we'll pretend for now.

Map II

```
nats = [1..]
square x = x * x
squares = map square nats
take 10 squares
```

Map III

Alternatively, we could have implemented this as:

```
myMap f [] = []
myMap f x:xs = (x*x):(myMap f xs)
```

Map IV

You might think of this as being, basically

```
final List<Integer> nats = //...
final List<Integer> result = new ArrayList<>();
for(final Integer i : nats){
    result.add(i * i);
}
return result;
```

Map V

In other words, we can say that `map` abstracts the concept of doing something to every element in a list* in place.

Higher Order Functions

We can, similarly, abstract away other common patterns:

- filtering elements out of a list
- combining all elements in a list
- ...and more

Filter

```
nats = [1..]  
isEven x = mod x 2 == 0  
evens = filter isEven nats
```

Fold

```
xs = [1,2,3,4]  
sum = foldl (+) 0 xs
```

The Lie

We said these functions operate on *lists* of elements, but actually they can operate on other types of data as well.

Elementary Logic

Formalizing Language

Our ultimate goal is to be able to rigorously investigate properties and characteristics of **abstract models**.

English is a terrible language for this task.

Accordingly, we must develop a way to formalize the concepts we wish to discuss.

Propositions

Definition: A *proposition* is a statement that is either true or false.

Propositions II

Which of the following are propositions?

- We are currently in CSC-300
- $2+2=4$
- Pigs have wings
- If pigs have wings, then I have a million dollars

Propositions III

To make it easier to work with propositions, we will typically use variables to represent them.

Traditionally, we use the variables $p, q, r, \dots$

Truth Tables

When working with propositions, we will often construct **truth tables**.

For a simple proposition p we have the truth table:

p
<i>true</i>
<i>false</i>

Negation

p	$\neg p$
<i>true</i>	<i>false</i>
<i>false</i>	<i>true</i>

Conjunction (and)

p	q	$p \wedge q$
<i>true</i>	<i>true</i>	<i>true</i>

p	q	$p \wedge q$
<i>true</i>	<i>false</i>	<i>false</i>
<i>false</i>	<i>true</i>	<i>false</i>
<i>false</i>	<i>false</i>	<i>false</i>

Disjunction (or)

p	q	$p \vee q$
<i>true</i>	<i>true</i>	<i>true</i>
<i>true</i>	<i>false</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>false</i>	<i>false</i>

Exclusive Disjunction (xor)

p	q	$p \oplus q$
<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>false</i>	<i>false</i>

Implication (if)

p	q	$p \rightarrow q$
<i>true</i>	<i>true</i>	<i>true</i>
<i>true</i>	<i>false</i>	<i>false</i>
<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>false</i>	<i>true</i>

If and only if (iff)

p	q	$p \leftrightarrow q$
<i>true</i>	<i>true</i>	<i>true</i>
<i>true</i>	<i>false</i>	<i>false</i>
<i>false</i>	<i>true</i>	<i>false</i>
<i>false</i>	<i>false</i>	<i>true</i>

Compound Propositions

We can also combine the above to create more complex propositions.

$$p \vee (q \wedge \neg p)$$

What is the value of **this** proposition?

Can this proposition be simplified?

Predicate Logic

Up to this point, we have been discussing *propositional logic*.

We will now turn to a more powerful system: *predicate logic*.

Predicates

A **predicate** is a statement that can be either true or false, *depending on its variable(s)*

For example: $P(n) = n$ is an even number

$P(2) = 2$ is an even number = *true*

Predicates II

We can also call $P(n)$ a **propositional function**.

In other words, it is a function that takes a variable and returns a truth value.

We would also say that n is the **subject** of the statement.

Quantifiers

Things get more interesting when we start to quantify our statements.

What does that mean?

Universal Quantification

If it's true that a predicate $P(x)$ is true for **all values of x** , then we say it is **universally quantified**.

We write this as $\forall x P(x)$.

We read this as "For all x , $P(x)$ "

Universal Quantification II

If we find an element for which $P(x)$ is **not** true, then we say that x is a **counterexample** of $\forall x P(x)$

Existential Quantification

If it's true that a predicate $P(x)$ is true for **at least one value of x** , then we say it is **existentially quantified**.

We write this as $\exists x P(x)$

We read this as “There exists an x such that $P(x)$ ”

Existential Quantification

Is it possible to have a counterexample for $\exists x P(x)$?

Domain of Discourse

One detail we have left out is, what kind of values are x ?

We typically need to state what the **domain** of the quantifier is when we use it.

For example, “In the domain of integers, $\forall x \text{Even}(2 \times x)$ ”

Equivalences

Statements involving predicates and quantifiers are logically equivalent iff they have the same truth value for all applications and domains

Equivalences II

equivalence	explanation
$\neg \forall x P(x) \equiv \exists x \neg P(x)$	De Morgan's
$\neg \exists x P(x) \equiv \forall x \neg P(x)$	De Morgan's
$\forall x (P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$	distribution
$\exists x (P(x) \vee Q(x)) \equiv \exists x P(x) \vee \exists x Q(x)$	distribution

Negation

When negating nested quantifiers, just apply De Morgan's to **one quantifier at a time**.

$$\begin{aligned}\neg \forall w \exists x \forall y \exists z P(x) &\equiv \exists w \neg \exists x \forall y \exists z P(x) \\ &\equiv \exists w \forall x \neg \forall y \exists z P(x) \\ &\equiv \exists w \forall x \exists y \neg \exists z P(x) \\ &\equiv \exists w \forall x \exists y \forall z \neg P(x)\end{aligned}$$

So, we just flip the quantifiers?

Negation II

$$\neg \forall x (P(x) \wedge \exists y Q(y)) \not\equiv \exists x (P(x) \wedge \forall y \neg Q(y))$$

$$\neg \forall x (P(x) \wedge \exists y Q(y)) \not\equiv \exists x (\neg P(x) \wedge \forall y \neg Q(y))$$

Just naively flipping quantifiers doesn't work!

Negation III

$$\neg \forall x (P(x) \wedge \exists y Q(y)) \equiv \exists x \neg (P(x) \wedge \exists y Q(y))$$

$$\equiv \exists x (\neg P(x) \vee \neg \exists y Q(y))$$

$$\equiv \exists x (\neg P(x) \vee \forall y \neg Q(y))$$

So it's best to think about it as a step by step process!

A Taste of Prolog

Logical Programming

The fundamental idea behind logical programming is to allow programmers to simply *describe* a problem using *predicate logic* instead of having to *tell the computer what to do*.

Declarative Programming

In other words, the programmer simply *declares* what output is desired, and, somehow, the output will be generated without the programmer needing to explain *how* this is done.

Logical Programming II

So, logical programming is simply declarative programming where the description of the solution is provided using logical statements.

Logical Programming III

Unfortunately, no one knows how to really do this in an efficient manner, so the programmer has to deal with certain constraints and will often need to give the system “hints” about how to execute programs.

Clausal Form

First, let’s talk about clauses. A statement in clausal form has the following structure:

$$P \vee Q \vee R \leftarrow A \wedge B \wedge C$$

In other words, a disjunction on the left and a conjunction of the right.

Clausal Form II

All predicate logic statements can be converted to clausal form, which also happens to be easier to manipulate in an automated manner.

(Though still *too* complicated!)

Resolution

Once we have a description in clausal form, we can use *resolution* to generate facts. For instance, let’s consider the following clauses:

$$\begin{aligned} \text{older}(\text{jan}, \text{joe}) &\leftarrow \text{mother}(\text{jan}, \text{joe}) \\ \text{wiser}(\text{jan}, \text{joe}) &\leftarrow \text{older}(\text{jan}, \text{joe}) \end{aligned}$$

Resolution II

First, we combine all *consequents* on the left and all *antecedents* on the right:

$$older(jan, joe) \vee wiser(jan, joe) \leftarrow mother(jan, joe) \wedge older(jan, joe)$$

Resolution III

We can then cancel out common propositions on both sides:

$$wiser(jan, joe) \leftarrow mother(jan, joe)$$

Now, if $mother(jan, joe)$ is known to be true, then we can trivially infer that $wiser(jan, joe)$ is true.

Resolution IV

(This, ignores *a lot* of details for how we can do things like incorporate variables)

Horn Clauses

As mentioned above, such inferences are still too complicated to be automated, in general.

Accordingly, we will be working with a more restricted form of clauses: **horn clauses**.

Horn Clauses II

A horn clause is a proposition in clausal form which, additionally, can only have zero or one proposition on the left hand side. In other words:

$$P \leftarrow A \wedge B \wedge C \wedge \dots$$

Horn Clauses III

We refer to the left hand side of a horn clause as the “head” and the right hand side as the “body”

A “headless” horn clause will be referred to as a *fact*.

Prolog

Prolog, then, is a programming language which allows programmers to describe a problem using horn clauses and have results computed in a (more or less) automated manner.

Prolog II

A *program* in Prolog consists of *facts*, *rules*, and a *goal*.

Facts

A Prolog program will have a *database* of headless horn clauses which provide the *facts* of a problem. For example:

$$mother(jan, joe)$$

Rules

A Prolog program will also contain *inference rules* which can be used to compute *new* facts. For example, given the above fact and the following rules:

$$\begin{aligned} older(jan, joe) &\leftarrow mother(jan, joe) \\ wiser(jan, joe) &\leftarrow older(jan, joe) \end{aligned}$$

We can infer the *fact* that *wiser(jan, joe)*

Goal

A *goal* is a headless horn clause which we want to “prove.” If, by applying resolution to the facts and rules of the program, we generate the *goal*, then that goal is “proven.” For example, the goal,

$$wiser(jan, joe)$$

would evaluate to *yes*.

Goal II

More interestingly, we can have *variables* in a goal. When the goal is “proven,” the variables will capture the values for which the proposition was true. For example:

wiser(jan, X)

would return $X = \text{joe}$.

Goal III

If there are *multiple* variable instantiations which would make the goal true, then they can all be returned by the system.

Prolog III

So, a complete prolog program might look like:

```
mother(jan,joe).
```

```
older(jan,joe) :- mother(jan,joe).
```

```
wiser(jan,joe) :- older(jan,joe).
```

```
?-wiser(X,Y).
```

which will generate all pairs of (X, Y) where X is *wiser* than Y (according to the information available)!

Prolog IV

Of course, it's possible that there are other pairs of people, such that one is wiser than the other, but it's important to emphasize that Prolog can only infer from the available facts and rules.

This is the “closed world” assumption.

Variables

Of course, we would probably want to generalize our rules to get the following program:

```
mother(jan,joe).  
  
older(X,Y) :- mother(X,Y).  
wiser(X,Y) :- older(X,Y).  
  
?-wiser(X,Y).
```

Variables II

Please pay special attention to the fact that these variables are more like variables in math, not in a typical programming language.

Evaluation

Prolog programs can be evaluated in various ways, conceptually.

Forward/Backward Chaining

One possibility is to evaluate from facts and rules until the goal is reached (forward chaining).

Another possibility is to work from the goal to the facts (backward chaining).

Depth/Breadth First

Also, the search for matches can be performed in either a breadth first or depth first manner.

Prolog

Either way, the evaluation is, essentially, an exhaustive tree search for matching values.

As you might imagine, this can be quite inefficient!

Backtracking

This also means that the evaluation has to keep track of solutions that have been attempted so that it can *backtrack* and reinitialize variables for subgoals to try and find a new path.

Arithmetic

We can also define arithmetic relationships between values in predicates.

```
min(10).  
max(20).  
  
avg(X):-min(A),min(B),X is (A+B)/2.
```

Lists

We can also have lists in our predicates. Lists consist of heads and tails, much like in Haskell and we can do similar types of pattern matching.

```
member(X, [X|_]).  
member(X, [_|XS]):-member(X,XS).
```

Purity

As mentioned previously, efficiency is a big problem with logic programming. Accordingly, Prolog diverges from being a pure logic language.

Order

One thing Prolog allows is for programmers to put facts and rules in a given order to enforce the order of evaluation. This allows the programmer to influence the run time by reducing the amount of backtracking necessary.

Cuts

Prolog also allows the programmer to include the predicate **!** (called a “cut”), which prevents back tracking for any predicate to its left.

Cuts II

For instance, if we know a predicate will only have one match, we can cut after that predicate to avoid wasting time finding another match.

Negation

Prolog is capable of proving that things are *true*, but it struggles proving things are *not* true. There are limited cases in which some sense of negation can be used, but it gets complicated quickly.

General Limitations

In general, there are simply many problems which can not be captured in predicate logic which *and* efficiently evaluated. In many cases, every possibility must simply be tested until the proper values are found.

Termination

Another problem, related to the previous, is that it's easy to write programs which simply will not terminate, as they can continue to generate more and more facts forever.

Practical Use

Unfortunately, the previously discussed limitations make using prolog for general purpose programming problematic.

It *has* been successful as a database query language, for expert systems, and for natural language processing.

Datalog

A similar logic that has had a lot of recent interest (again) is **Datalog**. Datalog appears very similar to Prolog, but has additional restrictions which allow it to have guaranteed polynomial run times and guaranteed termination.

Datalog II

Datalog can be embedded in programs written in other programming languages to allow for elegantly defined computations, with guarantees about safety.

Warning

I am a Datalog fanatic and may be very biased...